

Lab 6: Process Scheduling

1. Write a program in C language to implement FCFS process scheduling algorithm.

Source code:

```
#include <stdio.h>
int main() {
    int n, i, j, burst_time[20], arrival_time[20], wait_time[20], turnaround_time[20], completion_time[20];
    float avg_wait_time = 0, avg_turnaround_time = 0;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the arrival time and burst time for each process:\n");
    for (i = 0; i < n; i++) scanf("%d %d", &arrival_time[i], &burst_time[i]);
    for (i = 0; i < n; i++) for (j = i + 1; j < n; j++) if (arrival_time[i] > arrival_time[j]) {
        int temp = arrival_time[i]; arrival_time[i] = arrival_time[j]; arrival_time[j] = temp;
        temp = burst_time[i]; burst_time[i] = burst_time[j]; burst_time[j] = temp;
    }
    for (i = 0; i < n; i++) completion_time[i] = (i == 0 || arrival_time[i] > completion_time[i - 1]) ?
        arrival_time[i] + burst_time[i] : completion_time[i - 1] + burst_time[i];
    for (i = 0; i < n; i++) {
        turnaround_time[i] = completion_time[i] - arrival_time[i];
        wait_time[i] = turnaround_time[i] - burst_time[i];
        avg_wait_time += wait_time[i];
        avg_turnaround_time += turnaround_time[i];
    }
    avg_wait_time /= n; avg_turnaround_time /= n;
    printf("\nProcess\tArrival Time\tBurst Time\tWaiting Time\tTurnaround Time\tCompletion Time\n");
    for (i = 0; i < n; i++) printf("%d\t%d\t%d\t%d\t%d\t%d\n", i + 1, arrival_time[i], burst_time[i], wait_time[i], turnaround_time[i], completion_time[i]);
    printf("\nAverage waiting time: %.2f", avg_wait_time);
    printf("\nAverage turnaround time: %.2f", avg_turnaround_time);
}
```

Output:

```

PS C:\Users\samir\Desktop\project> cd "c:\Users\samir\Desktop\project\" ; if ($?) { gcc lab6.c -o lab6 } ; if ($?) { .\lab6 }
Enter the number of processes: 5
Enter the arrival time and burst time for each process:
Process 1: 5 5
Process 2: 3 4
Process 3: 4 4
Process 4: 1 3
Process 5: 2 7

Process Arrival Time    Burst Time    Waiting Time    Turnaround Time    Completion Time
1      1              3              0              3              4
2      2              7              2              9              11
3      3              4              8              12              15
4      4              4              11             15              19
5      5              5              14             19              24

Average waiting time: 7.00
Average turnaround time: 11.60
PS C:\Users\samir\Desktop\project>

```

2. Write a program in C to implement SJF process scheduling algorithm.

source code:

```

#include <stdio.h>
int main() {
    int n, i, j, smallest_index, time = 0, completed = 0;
    float avg_wait_time = 0, avg_turnaround_time = 0;
    int burst_time[20], arrival_time[20], wait_time[20], turnaround_time[20], completion_time[20], remaining_time[20];
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the arrival time and burst time for each process:\n");
    for (i = 0; i < n; i++) {
        scanf("%d %d", &arrival_time[i], &burst_time[i]);
        remaining_time[i] = burst_time[i];
    }
    for (i = 0; i < n; i++) for (j = i + 1; j < n; j++) if (arrival_time[i] > arrival_time[j]) {
        int temp = arrival_time[i]; arrival_time[i] = arrival_time[j]; arrival_time[j] = temp;
        temp = burst_time[i]; burst_time[i] = burst_time[j]; burst_time[j] = temp;
        temp = remaining_time[i]; remaining_time[i] = remaining_time[j]; remaining_time[j] = temp;
    }
    while (completed != n) {
        smallest_index = -1;
        for (i = 0; i < n; i++) if (remaining_time[i] > 0 && arrival_time[i] <= time && (smallest_index == -1 || remaining_time[i] < remaining_time[smallest_index])) smallest_index = i;
        if (smallest_index == -1) { time++; continue; }
    }
}

```

```

        remaining_time[smallest_index]--;
        if (remaining_time[smallest_index] == 0) {
            completion_time[smallest_index] = time + 1;
            turnaround_time[smallest_index] = completion_time[smallest_index] - arrival_time[smallest_index];
            wait_time[smallest_index] = turnaround_time[smallest_index] - burst_time[smallest_index];
            completed++;
        }
        time++;
    }
    for (i = 0; i < n; i++) {
        avg_wait_time += wait_time[i];
        avg_turnaround_time += turnaround_time[i];
    }
    avg_wait_time /= n; avg_turnaround_time /= n;
    printf("\nProcess\tArrival Time\tBurst Time\tWaiting Time\tTurnaround Time\tCompletion Time\n");
    for (i = 0; i < n; i++) printf("%d\t%d\t%d\t\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, arrival_time[i], burst_time[i], wait_time[i], turnaround_time[i], completion_time[i]);
    printf("\nAverage waiting time: %.2f\nAverage turnaround time: %.2f\n", avg_wait_time, avg_turnaround_time);
    return 0;
}

```

Output:

```

Enter the number of processes: 5
Enter the arrival time and burst time for each process:
Process 1: 0 2
Process 2: 2 3
Process 3: 4 5
Process 4: 6 6
Process 5: 7 9

Process Arrival Time    Burst Time    Waiting Time    Turnaround Time    Completion Time
1      0              2              0              2              2
2      2              3              0              3              5
3      4              5              1              6              10
4      6              6              4              10             16
5      7              9              9              18             25

Average waiting time: 2.80
Average turnaround time: 7.80
PS C:\Users\samir\Desktop\project>

```

3. Write a program in C to implement Round Robin process scheduling algorithm.

source code:

```

#include <stdio.h>
int main()
{
    int n, i, j, time_quantum, remaining_processes, time = 0;
    float avg_wait_time = 0, avg_turnaround_time = 0;
    int burst_time[20], arrival_time[20], wait_time[20], turnaround_time
[20],
        completion_time[20], remaining_time[20];
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the arrival time and burst time for each process:\n");
    for (i = 0; i < n; i++)
    {
        printf("Process %d:\n", i + 1);
        printf("Arrival time: ");
        scanf("%d", &arrival_time[i]);
        printf("Burst time: ");
        scanf("%d", &burst_time[i]);
        remaining_time[i] = burst_time[i];
    }
    printf("Enter the time quantum: ");
    scanf("%d", &time_quantum);
    // Calculate completion time and turnaround time for each process
    remaining_processes = n;
    while (remaining_processes > 0)
    {
        for (i = 0; i < n; i++)
        {
            if (remaining_time[i] > 0 && arrival_time[i] <= time)
            {
                if (remaining_time[i] > time_quantum)
                {
                    time += time_quantum;
                    remaining_time[i] -= time_quantum;
                }
                else
                {
                    time += remaining_time[i];
                    completion_time[i] = time;
                    turnaround_time[i] = completion_time[i] - arrival_time[i];

                    wait_time[i] = turnaround_time[i] - burst_time[i];
                    avg_wait_time += wait_time[i];
                    avg_turnaround_time += turnaround_time[i];
                    remaining_time[i] = 0;
                    remaining_processes--;
                }
            }
        }
    }
}

```

```

    }
    }
}

// Print results
avg_wait_time /= n;
avg_turnaround_time /= n;
printf("\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround
Time\tWaiting Time\n");
for (i = 0; i < n; i++) {
    printf("%d\t%d\t\t%d\t\t%d\t\t%d\t\t%d\n", i + 1, arrival_time
[i], burst_time[i],
        completion_time[i], turnaround_time[i], wait_time[i]);
}
printf("\nAverage Waiting Time: %.2f\n", avg_wait_time);
printf("Average Turnaround Time: %.2f\n", avg_turnaround_time);
return 0;
}

```

Output:

```

PS C:\Users\samir\Desktop\project> cd "c:\Users\samir\Desktop\project\" ; if ($?) { gcc lab6.c -o lab6 } ; if ($?) { .\lab6 }
Enter the number of processes: 4
Enter the arrival time and burst time for each process:
Process 1:
Arrival time: 0
Burst time: 5
Process 2:
Arrival time: 1
Burst time: 4
Process 3:
Arrival time: 2
Burst time: 2
Process 4:
Arrival time: 4
Burst time: 1
Enter the time quantum: 2

Process Arrival Time    Burst Time    Completion Time Turnaround Time Waiting Time
1         0             5             12             12             7
2         1             4             11             10             6
3         2             2             6              4             2
4         4             1             7              3             2

Average Waiting Time: 4.25
Average Turnaround Time: 7.25
PS C:\Users\samir\Desktop\project>

```